Quiz 4 - Results

Attempt 1 of 1

Written Feb 10, 2026 5:01 PM - Feb 10, 2026 5:08 PM

Attempt Score **10 / 10 - 100 %**

Overall Grade (Highest Attempt) **10 / 10 - 100 %**

Question 1

1 / 1 point

Why is the dot product in self-attention scaled by $\sqrt{d_k}$?

- To normalize gradients across layers
- To reduce memory usage.
- ✓ To prevent softmax from becoming overly peaked for large dimensions.
- To enforce orthogonality between queries and keys

Question 2

1 / 1 point

During the forward pass, PyTorch builds a _____ that enables autograd for backpropagation.

- Backpropagation graph
- Tensor state graph

- ○ Tensor dependency table
- ✓◉ Dynamic computational graph

Question 3 1 / 1 point

Which of the following are responsibilities of a PyTorch Dataset class?

- ✓☑ Lazily loading data instead of loading everything into memory.
- ✓☑ Implementing `__getitem__` to return a single data sample.
- ✓☑ Implementing `__len__` to define the dataset size.
- ✓☐ Handling minibatching and shuffling.

Question 4 1 / 1 point

Which of the following properties give Transformers advantages over RNN-based models?

- ✓☑ Transformers allow parallel computation over sequence positions during training.
- ✓☐ Transformers always require fewer parameters than RNN-based models for the same task.
- ✓☐ Transformers reduce the asymptotic time complexity of sequence modeling compared to RNNs.
- ✓☑ Transformers can model long-range dependencies without relying on sequential hidden state propagation.

Question 5 1 / 1 point

Which statement about the computational complexity of multi-head self-attention is correct?

- ✓ Splitting into multiple heads preserves overall $O(n^2d)$ complexity.
- Each head has $O(n^2)$ complexity, so the total complexity is multiplied by the number of heads.
- Multi-head attention reduces time complexity from $O(n^2)$ to $O(n)$.
- Complexity is independent of sequence length

Question 6 1 / 1 point

Which of the following is true about the architectural difference between Transformer encoder blocks and decoder blocks?

- Encoder blocks apply masking to prevent attending to future tokens, while decoder blocks do not.
- Decoder blocks process tokens sequentially, whereas encoder blocks process the input sequence in parallel.
- ✓ Decoder blocks include an additional attention mechanism that attends to encoder outputs, while encoder blocks do not.
- Encoder blocks use full self-attention, while decoder blocks replace self-attention with cross-attention.

Question 7 1 / 1 point

Which of the following statements about positional encoding in Transformers are correct?

- ✓ Positional encoding enforces left-to-right processing of sequences.

- [x] Positional encoding is combined with token embeddings before being fed into the model.
- [x] Positional encoding is needed because self-attention alone does not capture token order.
- [x] Positional encoding is not needed in the decoder due to the masking technique.

Question 8 1 / 1 point

What characterizes self-attention in Transformers?

- [x] Queries, keys, and values are all derived from the same input sequence.
- [ ] Self-attention requires explicit positional encoding to compute attention weights.
- [ ] Queries are generated from the decoder, while keys and values are generated from the encoder.
- [ ] Self-attention enforces left-to-right dependency through recurrent connections.

Question 9 1 / 1 point

Which of the following statements about tensors involved in PyTorch's autograd system is correct?

- [ ] Leaf tensors must have a non-`None` `grad_fn` to participate in gradient computation.
- [x] A leaf tensor can participate in gradient computation even though its `grad_fn` is None.

- Tensors without `requires_grad=True` are excluded from the computation graph and ignored during backpropagation.
- Leaf tensors are defined as tensors that do not depend on any other tensors.

Question 10 1 / 1 point

What is the primary purpose of multi-head attention in the Transformer architecture?

- To eliminate the need for positional encoding in sequence modeling.
- To enforce sequential processing of tokens during attention computation.
- To reduce the computational complexity of self-attention by splitting it into smaller heads
- ✓ To allow the model to attend to information from multiple representation subspaces simultaneously.

Done